



Node.js Events



Vinod Tahelyani



Read

Discuss

Courses

Practice

According to the official documentation of Node.js, it is an asynchronous event-driven JavaScript runtime. Node.js has an **event-driven architecture** which can perform asynchronous tasks. Node.js has 'events' module which emits named events that can cause corresponding functions or callbacks to be called.

Functions(Callbacks) listen or subscribe to a particular event to occur and when that event triggers, all the callbacks subscribed to that event are fired one by one in order to which they were registered.

The EventEmitter class: All objects that emit events are instances of the EventEmitter class. The event can be emitted or listen to an event with the help of EventEmitter.

Syntax:

```
const EventEmitter=require('events');  
var eventEmitter=new EventEmitter();
```

We use cookies to ensure you have the best browsing experience on our website. By using our site, you acknowledge that you have read and understood our [Cookie Policy](#) & [Privacy Policy](#).

Got It !

Syntax:



DSA
COURSE

"Kudos to
GeeksforGeeks!!!"



Sachin Motwani
Placed at- ORACLE

DSA Course: Success Story

The DSA self paced course is really helpful for performing well in a technical interview.

 24X7 Doubt Support

 10 Contests

 DSA Projects

[LEARN MORE](#)

```
eventEmitter.addListener(event, listener)
eventEmitter.on(event, listener)
eventEmitter.once(event, listener)
```

eventEmitter.on(event, listener) and **eventEmitter.addListener(event, listener)** are pretty much similar. It adds the listener at the end of the listener's array for the specified event. Multiple calls to the same event and listener will add the listener multiple times and correspondingly fire multiple times. Both functions return emitter, so calls can be chained.

eventEmitter.once(event, listener) fires at most once for a particular event and will be removed from listeners array after it has listened once. Returns emitter, so calls can be chained. **Emitting events:**

Every event is named event in nodejs. We can trigger an event by `emit(event, [arg1], [arg2], [...])` function. We can pass an arbitrary set of arguments to the listener functions.

Syntax:

```
eventEmitter.emit(event, [arg1], [arg2], [...])
```

Simple Event:

javascript

```
// Importing events
const EventEmitter = require('events');

// Initializing event emitter instances
var eventEmitter = new EventEmitter();

// Registering to myEvent
eventEmitter.on('myEvent', (msg) => {
  console.log(msg);
});

// Triggering myEvent
eventEmitter.emit('myEvent', "First event");
```

Output:

```
First event
```

Removing Listener: The **eventEmitter.removeListener()** takes two argument event and listener, and removes that listener from the listeners array that is subscribed to that event. While **eventEmitter.removeAllListeners()** removes all the listener from the array which are subscribed to the mentioned event.

Syntax:

```
eventEmitter.removeListener(event, listener)
eventEmitter.removeAllListeners([event])
```

Note:

- Removing the listener from the array will change the sequence of the listener's array, hence it must be carefully used.
- The **eventEmitter.removeListener()** will remove at most one instance of the listener which is in front of the queue.

javascript

```

// Importing events
const EventEmitter = require('events');

// Initializing event emitter instances
var eventEmitter = new EventEmitter();

var fun1 = (msg) => {
    console.log("Message from fun1: " + msg);
};

var fun2 = (msg) => {
    console.log("Message from fun2: " + msg);
};

// Registering fun1 and fun2
eventEmitter.on('myEvent', fun1);
eventEmitter.on('myEvent', fun1);
eventEmitter.on('myEvent', fun2);

// Removing listener fun1 that was
// registered on the line 13
eventEmitter.removeListener('myEvent', fun1);

// Triggering myEvent
eventEmitter.emit('myEvent', "Event occurred");

// Removing all the listeners to myEvent
eventEmitter.removeAllListeners('myEvent');

// Triggering myEvent
eventEmitter.emit('myEvent', "Event occurred");

```

Output:

```

Message from fun1: Event occurred
Message from fun2: Event occurred

```

We registered two times fun1 and one time fun2 for calling eventEmitter.removeListener('myEvent', fun1) one instance of fun1 will be removed. Finally, removing all listener by removeAllListeners() will remove all listeners to myEvent. **Other Methods:** By default, a maximum of 10 listeners can be registered for

any single event. To change the default value for all EventEmitter instances, the **EventEmitter.defaultMaxListeners** property can be used. The **eventEmitter.getMaxListeners()** will return the max listeners value set by **setMaxListeners()** or default value 10. **Note:** This is not a hard limit. EventEmitter will allow the new instance to be added but will print a warning message indicating possible EventEmitter memory leak.



Syntax:

```
eventEmitter.setMaxListeners(n)
eventEmitter.getMaxListeners()
```

javascript

```
// Importing events
const EventEmitter = require('events');

// Initializing event emitter instances
var eventEmitter1 = new EventEmitter();
var eventEmitter2 = new EventEmitter();

// Getting max listener
console.log("Default max listener for eventEmitter1 is: ",
    eventEmitter1.getMaxListeners());
console.log("Default max listener for eventEmitter2 is: ",
    eventEmitter2.getMaxListeners());
```

```

// Set global defaultMaxListeners to 2
EventEmitter.defaultMaxListeners = 2;

// Getting max listener
console.log("Default max listener for eventEmitter1 is: ",
            eventEmitter1.getMaxListeners());
console.log("Default max listener for eventEmitter2 is: ",
            eventEmitter2.getMaxListeners());

// Set max listener of eventEmitter1 to 5
eventEmitter1.setMaxListeners(5);

// Getting max listener
console.log("Default max listener for eventEmitter1 is: ",
            eventEmitter1.getMaxListeners());
console.log("Default max listener for eventEmitter2 is: ",
            eventEmitter2.getMaxListeners());

// Declaring listener fun1 to myEvent1
var fun1 = (msg) => {
    console.log("Message from fun1: " + msg);
};

// Declaring listener fun2 to myEvent2
var fun2 = (msg) => {
    console.log("Message from fun2: " + msg);
};

// Listening to myEvent1 with 3 instance of fun1

for(var i = 0; i < 3; i++) {
    eventEmitter1.addListener('myEvent1', fun1)
}

// Listening to myEvent2 with 3 instance of fun2
for(var i = 0; i < 3; i++){
    eventEmitter2.addListener('myEvent2', fun2)
}

// Emitting myEvent1 and myEvent2
eventEmitter1.emit('myEvent1', 'Event1 occurred');
eventEmitter2.emit('myEvent2', 'Event2 occurred');

```

Output:

```
Default max listener for eventEmitter1 is: 10
```

```
Default max listener for eventEmitter2 is: 10
Default max listener for eventEmitter1 is: 2
Default max listener for eventEmitter2 is: 2
Default max listener for eventEmitter1 is: 5
Default max listener for eventEmitter2 is: 2
Message from fun1: Event1 occurred
Message from fun1: Event1 occurred
Message from fun1: Event1 occurred
Message from fun2: Event2 occurred
Message from fun2: Event2 occurred
Message from fun2: Event2 occurred
(node:16240) MaxListenersExceededWarning: Possible
EventEmitter memory leak detected.
3 myEvent2 listeners added. Use emitter.setMaxListeners()
to increase limit
```

eventEmitter.listeners(): It returns an array of listeners for the specified event.

Syntax:

```
eventEmitter.listeners(event)
```

eventEmitter.listenerCount(): It returns the number of listeners listening to the specified event.

Syntax:

```
eventEmitter.listenerCount(event)
```

eventEmitter.prependOnceListener(): It will add the one-time listener to the beginning of the array.

Syntax:

```
eventEmitter.prependOnceListener(event, listener)
```

eventEmitter.prependListener(): It will add the listener to the beginning of the array.

Syntax:

```
eventEmitter.prependListener(event, listener)
```

javascript

```
// Importing events
const EventEmitter = require('events');

// Initializing event emitter instances
var eventEmitter = new EventEmitter();

// Declaring listener fun1 to myEvent1
var fun1 = (msg) => {
    console.log("Message from fun1: " + msg);
};

// Declaring listener fun2 to myEvent2
var fun2 = (msg) => {
    console.log("Message from fun2: " + msg);
};

// Listening to myEvent with fun1 and fun2
eventEmitter.addListener('myEvent', fun1);

// fun2 will be inserted in front of listeners array
eventEmitter.prependListener('myEvent', fun2);

// Listing listeners
console.log(eventEmitter.listeners('myEvent'));

// Count the listeners registered to myEvent
console.log(eventEmitter.listenerCount('myEvent'));

// Triggering myEvent
eventEmitter.emit('myEvent', 'Event occurred');
```

Output:

```
[ [Function: fun2], [Function: fun1] ]
2
Message from fun2: Event occurred
Message from fun1: Event occurred
```


Special Events: All EventEmitter instances emit the event **'newListener'** when new listeners are added and **'removeListener'** existing listeners are removed.

- **Event: 'newListener'** The EventEmitter instance will emit its own 'newListener' event before a listener is added to its internal array of listeners. Listeners registered for the 'newListener' event will be passed to the event name and reference to the listener being added. The event 'newListener' is triggered before adding the listener to the array.

```
eventEmitter.once( 'newListener', listener)
eventEmitter.on( 'newListener', listener)
```

- **Event: 'removeListener'** The **'removeListener'** event is emitted after a listener is removed.

```
eventEmitter.once( 'removeListener', listener)
eventEmitter.on( 'removeListener', listener)
```

- **Event: 'error'** When an error occurs within an EventEmitter instance, the typical action is for an **'error'** event to be emitted. If an EventEmitter does not have at least one listener registered for the 'error' event, and an 'error' event is emitted, the error is thrown, a stack trace is printed, and the Node.js process exits.

```
eventEmitter.on('error', listener)
```

javascript

```

// Importing events
const EventEmitter = require('events');

// Initializing event emitter instances
var eventEmitter = new EventEmitter();

// Register to error
eventEmitter.on('error', (err) => {
    console.error('whoops! there was an error');
});

// Register to newListener
eventEmitter.on( 'newListener', (event, listener) => {
    console.log(`The listener is added to ${event}`);
});

// Register to removeListener
eventEmitter.on( 'removeListener', (event, listener) => {
    console.log(`The listener is removed from ${event}`);
});

// Declaring listener fun1 to myEvent1
var fun1 = (msg) => {
    console.log("Message from fun1: " + msg);
};

// Declaring listener fun2 to myEvent2
var fun2 = (msg) => {
    console.log("Message from fun2: " + msg);
};

// Listening to myEvent with fun1 and fun2
eventEmitter.on('myEvent', fun1);
eventEmitter.on('myEvent', fun2);

// Removing listener
eventEmitter.off('myEvent', fun1);

// Triggering myEvent
eventEmitter.emit('myEvent', 'Event occurred');

// Triggering error

```

Output:

```
The listener is added to removeListener
The listener is added to myEvent
The listener is added to myEvent
The listener is removed from myEvent
Message from fun2: Event occurred
whoops! there was an error
```

Asynchronous events: The EventEmitter calls all listeners synchronously in order to which they were registered. However, we can perform asynchronous calls by using **setImmediate()** or **process.nextTick()**.

javascript

```
// Importing events
const EventEmitter = require('events');

// Initializing event emitter instances
var eventEmitter = new EventEmitter();

// Async function listening to myEvent
eventEmitter.on('myEvent', (msg) => {
  setImmediate( () => {
    console.log("Message from async: " + msg);
  });
});

// Declaring listener fun to myEvent
var fun = (msg) => {
  console.log("Message from fun: " + msg);
};

// Listening to myEvent with fun
eventEmitter.on('myEvent', fun);

// Triggering myEvent
eventEmitter.emit('myEvent', "Event occurred");
```

Output:

```
Message from fun: Event occurred
```

Message from async: Event occurred

Reference: <https://nodejs.org/api/events.html>

Last Updated : 06 Apr, 2023



Similar Reads

1. Node.js Process Signal Events

2. Callbacks and Events in Node.js

3. Keyboard Events in ElectronJS

4. Mongoose Document Model.events API

5. Node.js Connect Mysql with Node app

6. Node.js Automatic restart Node.js server with nodemon

7. How to resolve 'node' is not recognized as an internal or external command error after installing Node.js ?

8. Top 3 Best Packages Of Node.js that you should try being a Node.js Developer

9. How to do pagination Node.js with MySQL ?

10. How to resolve unhandled exceptions in Node.js ?

Related Tutorials

1. Spring MVC Tutorial

2. Spring Boot Tutorial
